Академия ТОП

ДИПЛОМНАЯ РАБОТА

на тему:

«Разработка веб-приложения StyleFit — интернет-магазина одежды с подбором товаров с учётом типа фигуры пользователя»

Выполнила:

Идрисова Айсина Эрнэстовна

Группа:

Р511

Казань — 2026

ПРОЕКТНОЕ ЗАДАНИЕ

Тема: разработка веб-приложения StyleFit — интернет-магазина одежды с подбором товаров с учётом типа фигуры пользователя.

Цель работы

Разработать веб-приложение, которое предоставляет каталог одежды и поддерживает персонализированный подбор товаров на основе выбранных пользователем параметров.

Задачи работы

- изучить предметную область электронной торговли одеждой и принципы персонализации;
- проанализировать технологии клиентской и серверной веб-разработки;
- сформулировать функциональные и нефункциональные требования;
- спроектировать архитектуру приложения, структуру данных и API;
- реализовать клиентский интерфейс на HTML, CSS и JavaScript;
- использовать Python, Django и Django REST Framework для серверной части;
- описать механизм подбора и подготовить проверяемые сценарии тестирования;
- зафиксировать ограничения текущей версии и направления развития.

Объект и предмет

Объект — процесс выбора одежды в интернет-магазине. Предмет — программные средства представления каталога и персонализированного подбора товаров.

Средства разработки

В проекте используются HTML, CSS, JavaScript, Python, Django и Django REST Framework. Конкретный тип базы данных и окончательный состав моделей необходимо сверить с актуальными файлами настроек и исходным кодом.

Ожидаемый результат

Ожидается веб-приложение с каталогом, получением товарных данных через API и механизмом подбора. Полноту регистрации, корзины и оформления заказа необходимо подтвердить отдельной проверкой: наличие интерфейсных элементов само по себе не доказывает, что серверная логика завершена.

Этап	Результат
Аналитика	Предметная область, аналоги, требования
Проектирование	Архитектура, модели, API, сценарии
Разработка	Клиентская и серверная части
Проверка	Тест-кейсы и результаты фактических запусков
Документирование	Описание, выводы

СОДЕРЖАНИЕ

Введение……1
1 Теоретическая часть……..…….3
1.1 Электронная торговля одеждой и персонализация……..3
1.2 Анализ предметной области и аналогов……...….5
1.3 Технологии разработки веб-приложений……...…6
1.4 Качество, безопасность и доступность……..10
2 О заказчике и проекте……12
2.1 Общая характеристика StyleFit…….….12
2.2 Целевая аудитория и сценарии…….……...13
2.3 Требования к системе…….….14
2.4 Ограничения и критерии готовности……15
3 Проектирование и реализация……...…17
3.1 Архитектура приложения……..….17
3.2 Модели данных и каталог……...……18
3.3 Обмен данными……...……19
3.4 Алгоритм подбора……21
3.5 Клиентский интерфейс……..……..23
4 Тестирование программного продукта……25
4.1 Подход к тестированию……..…….25
4.2 Результаты автоматизированного тестирования……...25
4.3 Ручная проверка пользовательских сценариев…….…….25
Заключение…….…….27

ВВЕДЕНИЕ

Электронная торговля стала привычным способом выбора товаров. Пользователь может изучать ассортимент, сравнивать характеристики и знакомиться с фотографиями без посещения физического магазина. Однако большой каталог не всегда упрощает выбор: покупателю приходится самостоятельно учитывать размер, фасон, цвет, назначение и другие характеристики одежды.

Одним из способов улучшить процесс выбора является персонализация. В интернет-магазине она может выражаться в фильтрации по характеристикам, подборе по заданным параметрам или ранжировании товаров в соответствии с предпочтениями пользователя. Такой механизм должен оставаться понятным: покупатель должен знать, какие параметры используются и почему система показывает определённые позиции.

Проект StyleFit направлен на объединение каталога одежды и персонализированного подбора в одном веб-интерфейсе. Клиентская часть отвечает за представление данных и взаимодействие с пользователем, а серверная часть хранит сведения о товарах, обрабатывает запросы и предоставляет данные через API. Такое разделение позволяет развивать интерфейс и сервер независимо.

Актуальность проекта связана с практической задачей сделать навигацию по ассортименту более удобной. С учебной точки зрения проект позволяет применить технологии клиентской и серверной разработки, модели базы данных, сериализацию, маршрутизацию, административное управление каталогом и тестирование.

Цель дипломной работы — разработать веб-приложение StyleFit, предоставляющее каталог одежды и возможность персонализированного подбора на основе доступных параметров пользователя.

Для достижения цели необходимо изучить предметную область; рассмотреть технологии; сформулировать требования; спроектировать архитектуру и модели данных; описать API; реализовать клиентский интерфейс; определить логику подбора; подготовить тестовые сценарии; оценить ограничения и возможные улучшения.

Объект работы — процесс выбора одежды в электронной торговле. Предмет — программные средства реализации каталога и подбора товаров. В работе применяются анализ предметной области, сравнение типовых функций магазинов, проектирование компонентов, моделирование пользовательских сценариев и подготовка функциональных тест-кейсов.

Практическая значимость заключается в создании учебного продукта, который может стать основой для дальнейшего развития магазина: расширения каталога, улучшения рекомендаций, добавления персонального кабинета и подготовки безопасного развёртывания.

Работа состоит из трёх глав. Первая глава рассматривает теоретические основы и технологии. Во второй описываются проект, пользователи и требования. Третья посвящена архитектуре, данным, API, интерфейсу и проверке работоспособности.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Электронная торговля одеждой и персонализация

Интернет-магазин представляет собой информационную систему, которая публикует сведения о товарах и позволяет выполнять действия, связанные с их выбором. Типовая система включает каталог, карточку товара, поиск, фильтры, сортировку, корзину, оформление заказа и административные средства управления ассортиментом. Реальный состав зависит от масштаба проекта и бизнес-процессов.

Одежда — категория товаров, выбор которых зависит от совокупности характеристик. Покупателю могут быть важны размер, материал, цвет, сезон, фасон, назначение, цена и особенности посадки. Если сведения неполные или представлены непоследовательно, сравнивать изделия становится сложнее. Поэтому качество каталога зависит не только от интерфейса, но и от организации данных.

Каталог является центральным элементом магазина. Его качество определяется полнотой карточек, понятной навигацией, доступностью изображений и единообразием характеристик. Например, размерные обозначения должны быть согласованы, цена должна храниться в числовом поле, а изображения должны иметь корректные пути. Эти требования облегчают фильтрацию и уменьшают вероятность ошибок.

Персонализация — адаптация выдачи или взаимодействия под запрос конкретного пользователя. Она может реализовываться через явный выбор параметров, историю действий, правила соответствия или алгоритмы машинного обучения. Для учебного проекта разумно начинать с прозрачных правил, которые легко объяснить, протестировать и воспроизвести.

В StyleFit персонализация рассматривается как сопоставление выбранного параметра с характеристиками товара. Если модель содержит соответствующие

атрибуты, система может отобрать позиции, которые удовлетворяют выбранным условиям. Точный набор полей и правил необходимо описывать по фактической реализации, а не по предполагаемой схеме.

Рекомендация не является гарантией идеальной посадки одежды. Она зависит от полноты данных, точности характеристик товара и правил фильтрации. На посадку также влияют лекала, материал, производитель и индивидуальные предпочтения. Поэтому результат следует представлять как подсказку, а не как безусловное утверждение о подходящем размере.

Персонализация должна дополнять обычный просмотр каталога. Пользователь должен иметь возможность изменить выбор или отказаться от подбора. Не следует использовать формулировки, которые объявляют один тип фигуры правильным, а другой неправильным. Параметры нужны для навигации по ассортименту, а не для оценки внешности.

Для персонализации требуется согласованная структура данных. Если значения вводятся свободным текстом, возможны разные варианты написания одного и того же признака. Справочники и ограниченный набор значений облегчают сравнение и проверку. Система должна также определять поведение при отсутствии параметра и при пустом результате.

Объяснимость повышает доверие к рекомендациям. Если результат формируется простым правилом, интерфейс может сообщить, что подбор основан на выбранной категории или другом параметре. В более сложной версии можно учитывать несколько характеристик, однако каждое новое правило должно быть описано и покрыто тестами.

При проектировании следует учитывать качество исходных данных. Если часть карточек не имеет заполненного атрибута, фильтрация может исключать подходящие товары. Поэтому необходимо определить, какие поля обязательны, какие могут быть пустыми и как интерфейс отображает неполные сведения. Валидация должна обеспечивать согласованность данных.

1.2 Анализ предметной области и аналогов

Перед разработкой полезно определить распространённые функции интернет-магазинов одежды. К ним относятся просмотр списка товаров, открытие карточки, просмотр фотографий, выбор характеристик, фильтрация по категории и цене, поиск, корзина и оформление заказа. Дополнительные возможности могут включать рекомендации, избранное, отзывы, подбор комплектов и персональные подборки.

Для учебного StyleFit важно не копировать весь функционал крупных коммерческих платформ, а выбрать функции, соответствующие цели дипломной работы. Полноценная торговая система дополнительно требует оплаты, логистики, возвратов, уведомлений, защиты персональных данных, мониторинга и поддержки пользователей. Эти процессы могут выходить за рамки текущей реализации.

Сравнение аналогов следует выполнять по конкретным критериям: наличие каталога, фильтров, персонализации, удобство навигации, объяснимость рекомендаций и сложность реализации. Приведённая таблица является обобщённым сравнением типов решений, а не результатом инструментального тестирования конкретных коммерческих сайтов.

Тип решения	**Преимущества**	**Ограничения**	**Вывод для StyleFit**
Обычный интернет-магазин	Каталог и стандартная навигация	Подбор в основном выполняет покупатель	Добавить дополнительный сценарий подбора
Магазин с фильтрами	Сужает ассортимент по атрибутам	Зависит от полноты характеристик	Хранить признаки структурированно

Сервис рекомендаций	Может ускорить поиск	Сложные алгоритмы труднее объяснять	Начать с прозрачных правил
StyleFit	Объединяет каталог и подбор	Ограничен текущей реализацией	Подтверждать функции кодом и тестами

Анализ показывает, что отличительной особенностью StyleFit должна быть понятная связь между выбранными параметрами и результатом. При этом пользователь не должен терять возможность самостоятельно изучать каталог. Персонализированная выдача не должна быть единственным доступным сценарием.

Оценивать аналогичные решения полезно не только по внешнему виду. Качество магазина определяется корректностью данных, устойчивостью API, сообщениями об ошибках, доступностью с клавиатуры и на небольшом экране, а также проверкой прав доступа. Эти критерии включаются в требования к StyleFit.

1.3 Технологии разработки веб-приложений

1.3.1 HTML, CSS и JavaScript

HTML описывает структуру страницы: заголовки, навигацию, формы, изображения, карточки и кнопки. Семантическая разметка делает документ понятнее браузеру, вспомогательным технологиям и разработчику. Поля форм должны иметь подписи, изображения — осмысленный альтернативный текст, а интерактивные элементы — понятные названия.

CSS отвечает за визуальное оформление. С помощью таблиц стилей задаются шрифты, цвета, размеры, отступы, границы, сетки и состояния элементов. Для StyleFit использовалась спокойная фиолетово-голубая палитра.

Общие правила карточек позволяют сохранять единые интервалы, пропорции изображений и расположение характеристик.

JavaScript добавляет интерактивность: обрабатывает действия, отправляет запросы к API и обновляет содержимое страницы. При сетевом обмене может использоваться Fetch API. Клиентская часть должна учитывать ошибки сети, пустые списки и ответы, которые не соответствуют ожидаемому формату.

Разделение структуры, оформления и поведения облегчает поддержку проекта. HTML описывает элементы, CSS — внешний вид, а JavaScript — реакцию на действия. При смешивании всех обязанностей в одном крупном файле изменения становятся сложнее, поэтому код целесообразно организовывать по назначению.

Адаптивность достигается гибкими размерами, сетками и медиазапросами. Карточки могут размещаться в нескольких колонках на широком экране и в одной колонке на телефоне. Элементы управления должны оставаться доступными при увеличении масштаба и не зависеть только от наведения курсора.

1.3.2 Python, Django и Django REST Framework

Python — язык программирования общего назначения, применяемый для серверной логики и обработки данных. Его синтаксис ориентирован на читаемость, что удобно при разработке учебного приложения. Для веб-сервера язык обычно используется вместе с фреймворком, который организует маршрутизацию и формирование ответов.

Django — веб-фреймворк, предоставляющий маршруты URL, модели, шаблоны, административную панель, миграции и встроенные средства безопасности. Архитектура помогает разделить описание данных, обработку запросов и представление. Миграции фиксируют изменения структуры базы данных в последовательных файлах.

Административная панель Django позволяет управлять каталогом без разработки отдельного интерфейса для каждой операции. Но форма администрирования не гарантирует корректность записи: обязательные поля, модель, форма и ограничения базы должны быть согласованы. Ошибка HTTP 400 требует изучения сообщения и журнала, а не исправления наугад.

Django REST Framework расширяет Django средствами построения API. Сериализаторы представляют модели в формате ответа и проверяют входные данные. Представления обрабатывают запросы, а маршруты связывают URL с обработчиками. Благодаря этому клиент получает данные без необходимости знать устройство базы.

В процессе разработки StyleFit проверялись маршруты получения каталога товаров и справочника типов фигуры. Ответ HTTP 200 для конкретного запроса означает успешную обработку этого запроса в проверенной конфигурации, но не доказывает полноту каталога или корректность всех сценариев. Необходимы проверки содержимого ответа и поведения при ошибках.

1.3.3 API и протокол HTTP

API определяет, как одна программная часть обращается к другой. В веб-приложении браузер отправляет HTTP-запрос на адрес сервера и получает ответ. В ответе могут содержаться JSON-данные, HTML-документ, изображение или сообщение об ошибке.

Метод GET используется для получения представления ресурса, POST — для отправки данных или создания ресурса, PUT и PATCH — для изменения, DELETE — для удаления. Реальное поведение зависит от маршрута и его реализации. Клиент должен использовать методы в соответствии с контрактом API.

Коды состояния позволяют определить общий результат запроса. Код 200 обычно обозначает успешную обработку, 201 — создание ресурса, 400 —

некорректный запрос или ошибку валидации, 401 — отсутствие действительной аутентификации, 403 — запрет доступа, 404 — отсутствие ресурса, 500 — внутреннюю ошибку сервера. В интерфейсе не следует показывать пользователю необработанное внутреннее исключение.

API каталога должен возвращать поля, необходимые карточкам товаров. Справочник типов фигуры должен выдавать доступные варианты в согласованном формате. Маршрут рекомендаций должен принимать допустимые параметры и контролируемо обрабатывать отсутствие или ошибочное значение параметра.

При отладке проверяется не только код состояния, но и тело ответа, тип содержимого, количество записей и поведение при пустых данных. Успешный ответ может содержать пустой список, поэтому HTTP 200 сам по себе не гарантирует, что пользовательский сценарий выполнен правильно.

1.3.4 База данных и модели

База данных обеспечивает долговременное хранение сведений. Для каталога одежды могут понадобиться название, описание, цена, изображение, категория и дополнительные атрибуты. Поля должны отражать реальные требования проекта. Не следует включать их в дипломное описание как существующие, если их нет в исходном коде.

Реляционная модель позволяет связывать таблицы через первичные и внешние ключи. Товар может относиться к категории, а категория — содержать множество товаров. Нормализация уменьшает дублирование данных. При этом структура не должна становиться сложнее, чем требуется реальным сценариям.

В Django модель описывает поля и связи, а миграции синхронизируют модель со схемой базы данных. После изменения моделей создают миграции и применяют их к базе. Перед сдачей нужно убедиться, что миграции существуют, применены и соответствуют текущим моделям.

Изображения часто хранятся как отдельные файлы, а в базе данных сохраняется путь к ним. Доступность картинки зависит от наличия файла, настройки MEDIA_ROOT и MEDIA_URL, а также конфигурации сервера. Ошибка 404 может означать неправильный путь, отсутствующий файл или неправильную выдачу медиафайлов.

1.4 Качество, безопасность и доступность

Качество веб-приложения включает функциональную пригодность, удобство, надёжность, производительность, совместимость, сопровождаемость и безопасность. Для StyleFit важны корректное отображение каталога, предсказуемая работа API и понятная обработка ошибок.

Проверка данных должна выполняться на сервере, даже если клиентская форма содержит проверки. Клиентскую проверку можно обойти, поэтому она нужна для удобства, а серверная — для целостности данных. Входные значения следует проверять по типу, диапазону и допустимому набору.

Права доступа назначаются по принципу минимальных привилегий. Обычный пользователь не должен выполнять административные операции только потому, что знает URL. Сервер проверяет разрешения для каждой защищённой операции; скрытие кнопки в интерфейсе не заменяет эту проверку.

Пароли необходимо хранить только с использованием безопасных механизмов фреймворка. Секретный ключ и другие конфиденциальные настройки не должны попадать в публичный репозиторий. Секреты следует хранить в переменных окружения или локальной конфигурации, исключённой из контроля версий.

При разработке нужно учитывать защиту от инъекций, межсайтовых сценариев, подделки межсайтовых запросов и утечки конфиденциальных данных. Django предоставляет встроенные механизмы защиты, но они полезны

только при корректной настройке. Не следует отключать защитные проверки без понимания последствий.

Доступность интерфейса предполагает достаточный контраст, логичный порядок элементов, подписи полей и видимый фокус клавиатуры. Сообщения об ошибках должны быть понятными и располагаться рядом с соответствующим полем или в заметной области. Важно не терять введённые значения из-за единичной ошибки.

Производительность зависит от количества товаров, размеров изображений, количества запросов и структуры ORM-запросов. На небольшом учебном наборе проблемы могут быть незаметны. При росте каталога могут потребоваться пагинация, оптимизация запросов, сжатие изображений и кеширование справочных данных.

Тестирование должно охватывать нормальные сценарии, граничные значения и негативные случаи. Следует проверять корректные параметры, пропущенные поля, неверные идентификаторы, пустой каталог, отсутствующее изображение и недоступность сервера. Фактический результат записывают только после выполнения проверки.

2 О ЗАКАЗЧИКЕ И ПРОЕКТЕ

2.1 Общая характеристика StyleFit

StyleFit — учебный веб-проект в предметной области интернет-торговли одеждой. Условному владельцу магазина требуется представить ассортимент в удобном цифровом виде и помочь посетителю быстрее перейти от общего просмотра к выбору. Если реального внешнего заказчика нет, это следует прямо указать в окончательной версии работы.

Название StyleFit отражает идею соединения стиля и подбора. Основной сценарий состоит в просмотре товаров и использовании параметров для получения более подходящей выдачи. Проект не гарантирует идеальную посадку изделия и не заменяет профессиональную консультацию; он предоставляет дополнительный инструмент навигации.

Серверная часть реализуется на Django с Django REST Framework, клиентская — с использованием HTML, CSS и JavaScript. Сервер предоставляет API, а браузер запрашивает данные и показывает их пользователю. Такое разделение позволяет проверять серверную логику отдельно от оформления.

В процессе разработки использовались маршрут `/api/catalog/products/` для получения каталога и маршрут `/api/catalog/body-types/?gender=female` для получения справочника типов фигуры. Точный адрес рекомендаций, состав моделей и полный список полей следует подтвердить по актуальным файлам проекта перед сдачей.

В ходе разработки наблюдались проблемы с медиафайлами: запросы к некоторым изображениям из `media/products/` возвращали 404. Также сохранение товара через Django Admin приводило к HTTP 400. Эти проблемы нельзя считать исправленными без повторного тестирования, поэтому они перечислены как открытые технические вопросы.

2.2 Целевая аудитория и пользовательские сценарии

Основная аудитория — посетители, которые хотят просматривать одежду онлайн и получать подсказки при выборе. Для них важны ясная структура каталога, качественные изображения, понятные характеристики и возможность быстро изменить параметры подбора.

Дополнительная аудитория — администратор или сотрудник магазина, отвечающий за актуальность каталога. Ему необходимы инструменты добавления и редактирования товаров, корректное хранение изображений и контроль отображаемых сведений.

Сценарии должны позволять вернуться к обычному каталогу в любой момент. Персонализация дополняет просмотр, а не блокирует его. При отсутствии подходящих результатов интерфейс должен объяснять ситуацию и предлагать изменить параметры вместо показа пустого экрана без пояснений.

№	Сценарий	Ожидаемый результат
1	Пользователь открывает каталог	Показан список товаров или понятное состояние загрузки
2	Открывает карточку	Показаны доступные характеристики и изображение
3	Запрашивает справочник	Возвращаются допустимые варианты
4	Выбирает параметр	Выдача формируется по предусмотренным правилам
5	Администратор добавляет товар	Запись сохраняется при корректных данных

6	API возвращает ошибку	Интерфейс показывает понятное сообщение

2.3 Требования к системе

Функциональные требования описывают действия системы, а нефункциональные — качество их выполнения. Требования должны быть проверяемыми, однозначными и согласованными с фактической реализацией. Таблица ниже задаёт базовый перечень для проверки.

Код	Требование	Приоритет	Проверка
FR-01	Отображать каталог из серверных данных	Высокий	API и интерфейс
FR-02	Показывать сведения в карточке	Высокий	Сопоставление ответа и карточки
FR-03	Предоставлять справочник типов фигуры	Высокий	Проверка API
FR-04	Подбирать товары по доступным параметрам	Высокий	Проверка нескольких вариантов
FR-05	Управлять каталогом через админ-панель	Средний	Создание и редактирование

FR-06	Обрабатывать отсутствующее изображение	Средний	Тест пустого/неверного пути
FR-07	Показывать ошибки API	Средний	Сымитировать отказ сервера
FR-08	Регистрация, корзина и заказ	Требует подтверждения	Проверить серверные маршруты и сценарии

FR-08 требует отдельной сверки с текущей версией проекта. Наличие страниц или кнопок регистрации, корзины и заказа не доказывает, что серверная обработка завершена. В итоговом документе нужно оставить только функции, которые можно продемонстрировать и подтвердить исходным кодом.

К нефункциональным требованиям относятся адаптивное отображение, единая визуальная система, понятные сообщения, отсутствие секретов в репозитории, проверка входных данных на сервере, целостность каталога и сопровождаемость кода. При необходимости требования дополняются критериями времени ответа и доступности.

2.4 Ограничения и критерии готовности

Учебный проект не следует автоматически считать готовой коммерческой платформой. Для реального использования потребуются аудит безопасности, политика обработки персональных данных, резервное копирование, мониторинг, настройка производственного сервера и нагрузочное тестирование.

Критерии готовности должны быть конкретными. Например, маршрут каталога возвращает ожидаемую структуру, изображения доступны по

корректным адресам, а неверные данные приводят к контролируемому отказу. Формулировка «сайт работает хорошо» не даёт воспроизводимого результата.

Перед защитой следует проверить запуск проекта, установку зависимостей, миграции, создание административной учётной записи, наличие медиафайлов и основные маршруты. Если проект запускается только на компьютере разработчика, это необходимо указать как ограничение.

3 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ

3.1 Архитектура приложения

Архитектура StyleFit основана на разделении клиентской и серверной частей. Браузер отвечает за пользовательское взаимодействие и отображение данных. Django обрабатывает HTTP-запросы, обращается к моделям и формирует ответ. Django REST Framework организует сериализацию и API.

Разделение компонентов облегчает сопровождение. Изменение оформления кнопки не требует менять структуру базы, а изменение ответа API можно тестировать независимо от дизайна. При этом клиент и сервер должны согласовывать названия полей, формат ответов и способы обработки ошибок.

Логическая последовательность работы: пользователь → браузерный интерфейс → HTTP-запрос → маршрутизация Django/DRF → представление и сериализатор → модель и база данных → JSON-ответ → отображение результата. Медиафайлы выдаются отдельным механизмом, который зависит от настроек приложения и сервера.

Компонент	Назначение	Ответственность
Клиентский интерфейс	Взаимодействие с пользователем	Формы, карточки, загрузка, адаптивность
JavaScript	Поведение браузера	Запросы к API, обработка JSON
URLconf Django	Маршрутизация	Связь адреса и обработчика
Представления DRF	Обработка API	Получение и фильтрация данных
Сериализаторы	Формат и валидация	Поля ответа и проверка значений
Модели и БД	Хранение данных	Товары и справочники

Медиа-хранилище	Изображения	Файлы и URL изображений

Рисунок 3.1 — Логическая схема компонентов. Для итоговой версии рекомендуется заменить текстовое описание графической схемой, созданной в Word, Visio или draw.io, и сверить её с реальной структурой проекта.

3.2 Модели данных и организация каталога

Модель товара должна включать только поля, реально присутствующие в исходном коде. В типовом каталоге могут храниться название, описание, цена, изображение и категория. Дополнительные поля типа фигуры, цвета, размера или назначения добавляются только при наличии соответствующих требований и фактической реализации.

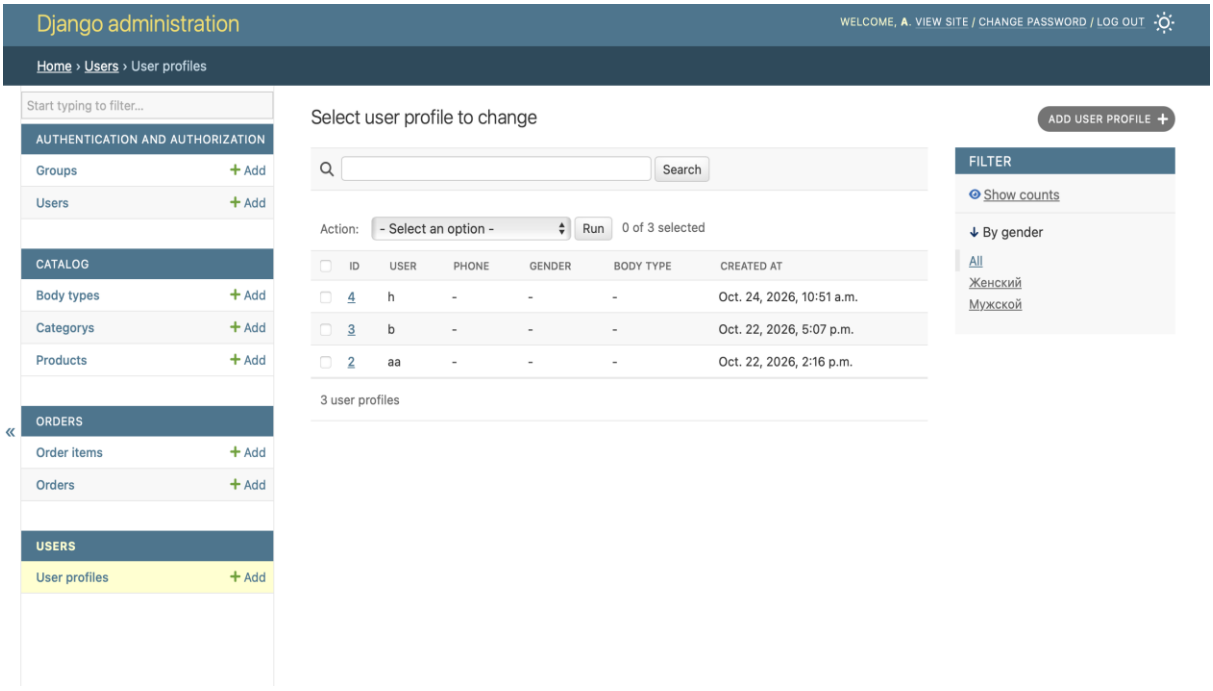
Справочник типов фигуры ограничивает набор допустимых значений и позволяет выдавать их через API. Если справочник зависит от параметра пола или другого выбора, фильтрация должна быть явно реализована и описана. Значения формы и атрибуты товаров должны совпадать по формату.

Связи между моделями выбираются по предметной области. Один товар может относиться к одной категории, а категория включать много товаров. Если товар подходит нескольким типам фигуры, модель должна поддерживать множественное соответствие. Нельзя утверждать, что такая связь уже реализована, пока она не проверена в моделях.

Цены следует хранить в числовом типе, подходящем для денежных значений, а не в произвольной строке. Изображение обычно хранится как файл, а в базе записывается путь. Поэтому корректность карточки зависит от сохранности файла и правильного URL.

Административная форма должна быть согласована с моделью. Ошибка сохранения может быть связана с обязательным полем, несовпадением типов,

валидацией, файлом изображения или ограничением базы. Для диагностики нужно изучить страницу ошибки и журнал сервера, затем проверить конкретное поле и повторить тест.



3.3 Обмен данными

Обмен данными в системе: принципы и сценарии

Обмен данными — это согласованный процесс передачи информации между компонентами системы (например, между бэкендом и фронтендом, между сервисами или между системой и внешними источниками). Даже без явного API обмен опирается на чёткие правила: формат, структуру, порядок и обработку ошибок.

Что делает обмен предсказуемым

- **Единые форматы данных.** Чаще всего используют JSON или XML: они структурированы, читаемы и поддерживаются большинством языков.
- **Чёткие схемы данных.** Это описание, какие поля ожидаются, какие из них обязательные, а какие — опциональные, и какие типы значений допустимы.

- **Согласованные состояния.** Система заранее определяет, как выглядит «успех», «ошибка», «пустые данные» и «временная недоступность».

Типичный сценарий обмена

1. **Инициация запроса.** Клиент (браузер, мобильное приложение, другой сервис) формирует запрос и отправляет его.
2. **Подготовка данных на стороне источника.** Система собирает нужные данные, приводит их к ожидаемому виду, отсекает лишнее.
3. **Передача.** Данные отправляются по каналу связи в заранее оговорённом формате.
4. **Приём и валидация.** Получатель проверяет, что структура соответствует ожиданиям, типы полей корректны, нет неожиданных полей.
5. **Обработка результата.** В зависимости от статуса система либо отображает данные, либо показывает понятное сообщение об ошибке, либо предлагает повторить действие.
6. **Обработка крайних случаев.** Отдельно обрабатываются ситуации «данных нет», «временная ошибка сети», «некорректный запрос».

Как избежать проблем при обмене

- **Не доверять входящим данным.** Даже если источник «свой», всегда проверять типы, диапазоны значений, обязательные поля.
- **Разделять «нет данных» и «ошибка».** Пустой список — это валидный результат, а не обязательно ошибка.
- **Показывать понятные состояния пользователю.** Вместо пустого экрана — сообщение: «Товары не найдены», «Попробуйте позже», «Проверьте фильтры».
- **Контролировать ошибки сети.** Временные сбои не должны ломать интерфейс: нужны индикаторы загрузки, сообщения и возможность повторить запрос.

- **Фиксировать поведение по умолчанию.** Если какой-то параметр не передан, система должна вести себя предсказуемо: например, вернуть все записи или применить стандартные фильтры.

3.4 Алгоритм персонализированного подбора

Алгоритм подбора сопоставляет выбранный параметр пользователя с атрибутами товаров. В простом варианте система получает значение из формы, проверяет его допустимость и выбирает записи, у которых соответствующее поле или связь содержит совпадающее значение.

Детерминированные правила подходят для учебной реализации, поскольку одинаковые входные данные приводят к одинаковому результату. Это облегчает объяснение и тестирование. Модели машинного обучения потребовали бы размеченного набора данных, оценки качества и дополнительных ресурсов.

Общая последовательность состоит из получения параметра, его проверки, получения доступных товаров, применения правил соответствия, формирования ответа и отображения результата. В зависимости от реализации шаги выполняются на сервере, в браузере или совместно. В дипломе нужно описать фактический вариант.

Если подбор использует несколько атрибутов, следует определить их приоритет и логику объединения. Например, один параметр может служить основным фильтром, а другой — дополнительным. Для каждой важной комбинации нужен тест, подтверждающий, что ожидаемые товары включены в результат, а неподходящие исключены.

Алгоритм должен корректно обрабатывать отсутствие параметра и пустую выдачу. Если значение не выбрано, интерфейс может предложить сначала выбрать вариант либо показать обычный каталог. Если значение неизвестно,

сервер должен вернуть контролируемую ошибку или безопасный пустой результат в соответствии с контрактом API.

Подбор не должен оценивать внешность пользователя. Тип фигуры используется как параметр навигации по ассортименту, а не как критерий привлекательности. Пользователь должен иметь возможность отказаться от персонализации и продолжить самостоятельный просмотр.

В будущем можно учитывать размер, цвет, цену, сезон, материал и предпочитаемый стиль. Каждый дополнительный фактор повышает требования к качеству карточек. Если атрибуты заполнены непоследовательно, даже хорошо написанный алгоритм будет выдавать неполные или неожиданные результаты.

Шаг	Вход	Действие	Выход
1	Параметр	Проверить допустимость	Значение или сообщение
2	Валидное значение	Получить товары	Набор записей
3	Товары и параметр	Применить правила	Отфильтрованный набор
4	Результаты	Сериализовать данные	Ответ API
5	Ответ API	Показать карточки или пустое состояние	Результат для пользователя

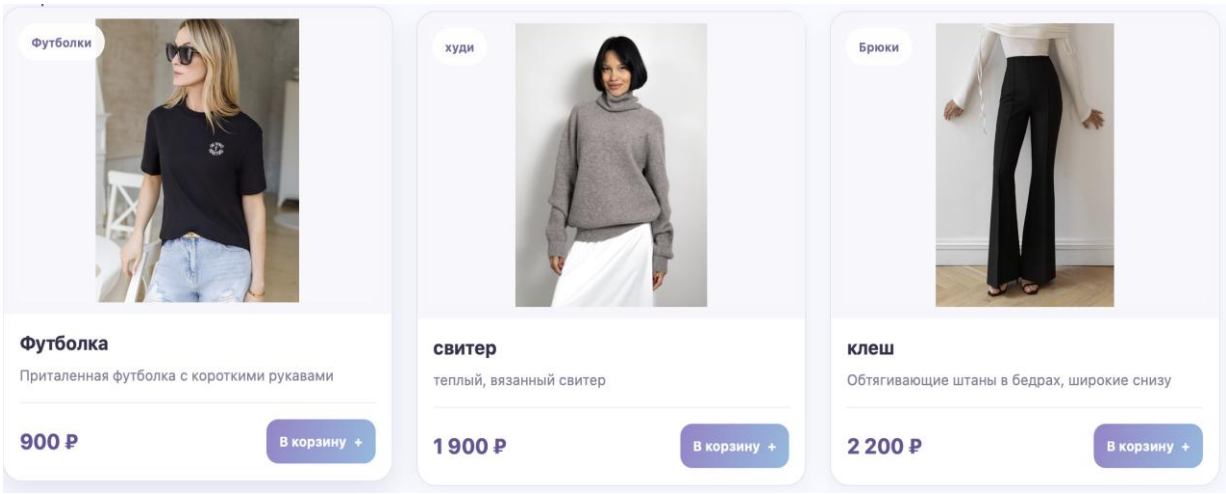
3.5 Клиентский интерфейс

Интерфейс StyleFit должен позволять понять структуру магазина без предварительного обучения. Заголовки, навигация, каталог и основные действия

должны различаться визуально. В карточке товара последовательно располагаются изображение, название, цена и доступные характеристики.

Единая визуальная система помогает сделать интерфейс целостным. Для проекта использовалась мягкая фиолетово-голубая палитра, скруглённые углы и карточная композиция. Цвет не должен быть единственным способом обозначения ошибки или успеха: требуется также текстовое пояснение или значок.

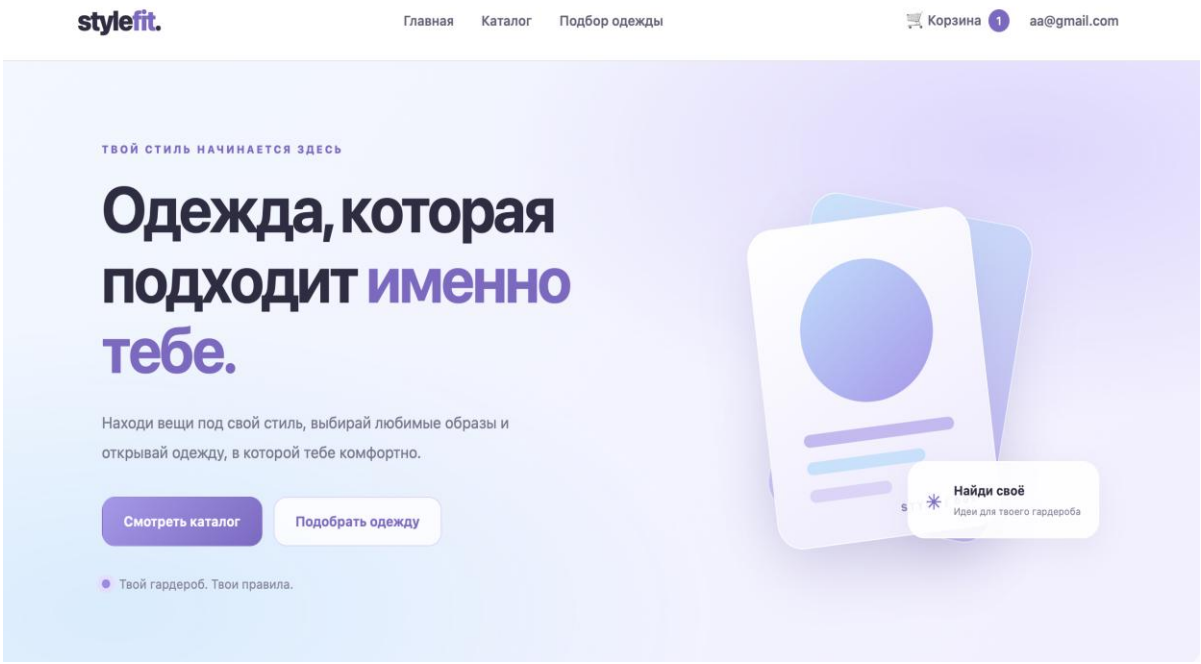
Карточки должны сохранять общую структуру при разной длине названий. Для изображений следует задавать предсказуемую область и режим отображения, чтобы фотографии не деформировались. Если файл отсутствует, нужно показывать нейтральную заглушку или сообщение вместо сломанного изображения.



Адаптивная сетка меняет число колонок в зависимости от ширины экрана. На мобильном устройстве элементы управления должны оставаться доступными, текст не должен выходить за границы карточки, а страница не должна требовать нежелательной горизонтальной прокрутки.

Формы требуют подписи полей, пояснения ожидаемого формата и обратной связи. При ошибке желательно сохранить введённые значения. Для запросов к серверу полезно показывать состояние загрузки, чтобы пользователь не нажимал кнопку несколько раз из-за отсутствия реакции.

Пользовательские сценарии должны соответствовать серверной функциональности. Если регистрация или корзина реализованы только визуально, их нельзя описывать как завершённые функции. В таком случае корректно писать, что интерфейс предусмотрен, а серверная обработка требует реализации.



4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

4.1 Подход к тестированию

Тестирование проводилось с использованием встроенного механизма Django TestCase и инструментария APITestCase из Django REST Framework. Тесты проверяют модели и обработку запросов на уровне серверной части. Для запуска использовалась команда python manage.py test внутри контейнера Docker. Django при запуске тестов создаёт отдельную временную тестовую базу данных. После завершения тестирования она удаляется. Поэтому обычный запуск тестов не должен изменять рабочие записи в основной базе данных проекта. Проверки были сгруппированы по приложениям catalog, users и orders. Такой подход позволяет сначала локализовать ошибки внутри отдельного компонента, а затем выполнить общий запуск всего набора тестов.

4.2 Результаты автоматизированного тестирования

В приложении catalog проверялись создание категорий, типов фигуры и товаров, строковое представление объектов, значения полей по умолчанию и связи между товарами и типами фигуры. В приложении users проверялись создание профиля, его строковое представление, связь с типом фигуры, регистрация пользователя, успешная авторизация, отказ при неправильном пароле, ограничение доступа к профилю и получение профиля авторизованным пользователем. В приложении orders проверялись создание заказа и позиции заказа, строковое представление, успешное оформление заказа, вычисление итоговой суммы, фиксация цены товара, уменьшение складского остатка, отказ при заказе неактивного товара, отказ при недостаточном остатке, ограничение списка заказов текущим пользователем и требование авторизации. Итоговый общий запуск показал: Found 25 test(s), далее все 25 проверок завершились успешно. В отчёте Django указано OK, а тестовая база данных была удалена после завершения проверки.

4.3 Ручная проверка пользовательских сценариев

Автоматизированные тесты не заменяют проверку интерфейса. Для комплексной оценки готового приложения необходимо дополнительно пройти пользовательские сценарии в браузере: открыть главную страницу, зарегистрировать нового пользователя, выполнить вход, открыть профиль, просмотреть каталог, выбрать товары, добавить их в корзину и оформить заказ. После оформления заказа следует проверить запись в административной панели, состав позиций, итоговую сумму, статус и изменение складского остатка. Отдельно необходимо проверить поведение при недостаточном количестве товара и при попытке открыть защищённую страницу без авторизации. В дипломную работу рекомендуется включить скриншоты фактической версии приложения: главная страница, каталог, карточка товара, регистрация и вход, профиль, корзина, форма оформления заказа, созданный заказ в административной панели, а также вывод успешного запуска тестов.

ЗАКЛЮЧЕНИЕ

В дипломной работе рассмотрена задача разработки веб-приложения StyleFit — интернет-магазина одежды с персонализированным подбором. Проект объединяет клиентский интерфейс и серверную часть, предназначенную для хранения и выдачи данных каталога.

В теоретической части рассмотрены особенности электронной торговли одеждой, роль персонализации, принципы API и применение HTML, CSS, JavaScript, Python, Django и Django REST Framework. Отдельное внимание уделено качеству данных, обработке ошибок и безопасности.

В проектной части описаны предполагаемые пользователи, основные сценарии и требования. Выделены ключевые функции: просмотр каталога, получение справочника типов фигуры, подбор по доступным параметрам и управление каталогом через административные средства.

В технической части представлена логическая архитектура, общий подход к моделям, взаимодействию через API и последовательности подбора. Подготовлены тест-кейсы для проверки маршрутов, карточек, изображений, административной панели и обработки ошибок.

Для окончательного подтверждения результата необходимо выполнить проверки на текущей версии проекта, проверить медиафайлы, разобраться с ошибкой сохранения товара, сверить модели и маршруты с исходным кодом и добавить реальные скриншоты интерфейса. Непроверенные функции нельзя описывать как полностью готовые.

Практическая значимость StyleFit состоит в том, что проект может развиваться от учебного каталога к более функциональному сервису. Перспективными направлениями являются расширение характеристик, улучшение алгоритма рекомендаций, автоматизация тестирования и безопасное развёртывание.

Таким образом, проект формирует основу для дальнейшей разработки и демонстрирует применение основных инструментов веб-программирования. Итоговая оценка готовности должна опираться на воспроизводимые проверки, а не только на описание предполагаемых возможностей.